Notes for C++

Section 7 : More Functions

Overview of Section 7

Week 7 is a catch-up week and continues with the important topic of Functions which starts in Section 6 of the notes available on Stream under Week 6.

Functions are an important part of programming and you are encouraged to design functions for your own programs and assignments. Assignments may not specifically ask for functions but you need to decide if it would be worthwhile to include functions. If an assignment (or quiz) specifically asks for a function, then make sure you include the function.

There is NOT a rule that says "every program you ever write must have a function in it".

Look under the heading "Why do we use functions?". If you are writing a program and you believe that it would be worth using a function, then go ahead and include a function. In particular, look at the notes about how "Functions enable us to test the main program at an early stage" and "Functions make a program more readable". Many people assume that the main purpose of functions is to avoid duplication of code. While it is useful to avoid duplication, this is not the main reason to use functions.

A reminder about Local and Global variables

Global variables are declared at the top of the program – main and all the functions know about the global variables and can access them.

Local variables (like parameters) exist only inside the function where they are declared – other functions cannot access them.

Why do we use local variables?

- Local variables make the function completely independent of main. The function has its own "world" with data coming in (as parameters) and any extra variables needed are local and completely separate from main.
- This really helps us when we are isolating a logic error inside a function as the function world has no connection to main.
- This also helps to make functions **portable**. If the function is completely separate from main we can confidently copy the function and use it in a new program (or put it in a library for other programmers to use).

Example: Showing the priority of local variables

Program A

```
#include <iostream>
using namespace std;
```

Program B

```
#include <iostream>
using namespace std;
```

```
void ping();

int a, b;

int main() {
  a = 3;
  b = 5;
  ping();
  cout << "a = " << a;
  cout << "  b = " << b;
}

void ping() {
  a = 27;
}
```

```
void pong();

int a, b;

int main() {
  a = 3;
  b = 5;
  pong();
  cout << "a = " << a;
  cout << "  b = " << b;
}

void pong() {
   int a;
   a = 27;
}
```

Answer – Program A displays: `a = 27 b = 5` and Program B displays: `a = 3 b = 5`

Note that Program B has a local variable. Thus the value 27 is loaded into the local variable and does not affect the global variable (which is loaded with 3). In the case of Program A, there is no local variable and so the function loads the value 27 into the global variable.

A program with functions – checking student course enrolment

Most students enrol in a degree that has some core (compulsory) courses. The university needs to keep track of how many core courses each student is taking. Write a program that works for just one student in one particular degree. (Advanced challenge: make this work for 100 students taking one of 3 different degrees, each with different core courses).

This program first establishes how many core courses there are in the degree (in this case there are three) and then loads the course numbers of the core courses into the array called `core`. Then the program loads the courses that the student is taking into the array called `courses`. Finally, the program counts how many core courses the student is doing and displays the result.

```
#include <iostream>
using namespace std;

bool checkcore(int corecrs[], int count, int coursenum);

int main() {
  int num, numcore, i, temp, corecount;
  int core[10], courses[40];

// section 1 : load the core courses
  numcore = 3;
  core[0] = 159101;
  core[1] = 161111;
  core[2] = 247112;

// section 2 : load the courses for one student
  i = 0;
  cout << "Enter a course for this student (-1 to stop) ";
  cin >> temp;
```

```
  while (temp != -1) {
    courses[i] = temp;
    i++;
    cout << "Enter a course for this student (-1 to stop) ";
    cin >> temp;
  }
  num = i;
  cout << "This student is taking " << num << " courses.\n";

// section 3 : count how many core courses
  corecount = 0;
  for (i = 0; i < num; i++) {
    if (checkcore(core, numcore, courses[i])) {
      corecount++;
    }
  }
  cout << corecount << " of the courses are core courses.\n";
}

bool checkcore(int corecrs[], int count, int coursenum) {
  int i;
  for (i = 0; i < count; i++) {
    if (coursenum == corecrs[i]) {
      return true;
    }
  }
  return false;
}
```

Interesting things to notice in this program:

- there are no global variables and all the variables are local.
- in this program it is very important for main and the function to each have their own variable called `i`. This is because `i` in the function gets changed many more times than the `i` in main.
- core courses are sent as an array parameter to the function
- only one element of courses is sent to the function. This is a normal integer parameter (not an array parameter) because the function only looks at one course at a time.
- the function is a **Boolean function** (it returns true or false). Boolean functions are very useful when something has to be checked – in this case, is this course a core course?

A program with functions – what day of the week is the ANZAC Day holiday?

Write a program to calculate what day of the week the ANZAC Day holiday will fall on.

Understand the problem:
What date is ANZAC day? is the holiday always on that date? (no, it is sometimes shifted to the following Monday.) If we know the day of the week for 01 April then ANZAC Day is 24 days later. Invent a code for the days of the week, for example 0 = Sun, 1 = Mon, 2 = Tues, 3 = Weds, 4 = Thurs, 5 = Fri, 6 = Sat (but you could invent a different code).

Create some examples:
Example 1: 01 April in 2019 is on a Monday (code = 1) and the holiday is on a Thursday (code = 4)
Example 2: 01 April in 2020 is on a Weds (code = 3) and the holiday is on a Monday (code = 1)
How many examples do you need? At least one example for each different situation that may arise.

Construct a solution:
The weekday code for ANZAC Day = code for 01 April + 24
But we are working with weeks (groups of 7 days) so ANZAC code = 01 April code + 24 % 7
Watch out for BEDMAS! Remainder is like division so we need brackets: (01 April code + 24) % 7
If ANZAC is a Sunday (code = 0) or Saturday (code = 6) then shift it to a Monday (code = 1)
***** TEST your solution using your examples *****

Write your program:
Use your examples to design the display.
While coding, invent new functions to hide details – keep your focus on main
Use comments when you need to remember why you have done something
Include extra DEBUG cout statements to display variables that are otherwise hidden

```
#include <iostream>
using namespace std;

int aprilstartday();
int anzacday(int day1);
void printday(int dayofweek);

int year, firstday, anzac;

int main() {
  cout << "Enter the year ";
  cin >> year;
  firstday = aprilstartday();
  cout << "April 01 in " << year << " is on a ";
  printday(firstday);
  cout << endl;
  anzac = anzacday(firstday);
  cout << "The ANZAC Day holiday in " << year << " is on a ";
  printday(anzac);
  cout << "\nDEBUG anzac = " << anzac;  // debug - remove later
  cout << endl;
}

int aprilstartday() {
  int temp;
  cout << "0 = Sun  1 = Mon  2 = Tues  3 = Weds";
  cout << "    4 = Thurs  5 = Fri  6 = Sat\n";
  cout << "Using this coding system, enter the day of the week ";
  cout << "for 01 April in " << year << endl;
  cout << "For example, if 01 April is on a Friday then enter 5\n";
  cin >> temp;
  return temp;
}

int anzacday(int day1) {
  int temp;
  temp = (day1 + 24) % 7;  // move from 01 to 25 April
  if ((temp == 0) || (temp == 6)) {
    temp = 1;       // holiday is on Monday if 25th is Sat or Sun
  }
```

```
  return temp;
}

void printday(int dayofweek) {
  if (dayofweek == 0) {
    cout << "Sunday";
  } else if (dayofweek == 1) {
    cout << "Monday";
  } else if (dayofweek == 2) {
    cout << "Tuesday";
  } else if (dayofweek == 3) {
    cout << "Wednesday";
  } else if (dayofweek == 4) {
    cout << "Thursday";
  } else if (dayofweek == 5) {
    cout << "Friday";
  } else if (dayofweek == 6) {
    cout << "Saturday";
  }
}
```

IMPORTANT SECTION: Why do we use functions?

- Functions can avoid duplication of code.
- Functions enable us to focus on one topic at a time.
- Functions can be used to isolate errors and thus assist with debugging.
- Functions enable us to test the main program at an early stage.
- Functions make a program more readable.
- We can save time and effort by using functions from previous programs when creating a new program (or by using functions that are available in a library).

Functions can avoid duplication of code

We can use a function several times in a program. If the function was not available, we would have to type in two sections of code that would be almost exactly the same. The function avoids this. Some people say that duplication of code is the only reason to use functions. You can see this opinion is not correct by looking through the rest of this list.

Functions enable us to focus on one topic at a time and thus assist with program design

For example, in the ANZAC Day program, the topic for the main program is "calculate the day of the week for the holiday". We do not want to get distracted by side issues such as "is the code for Sunday a 0 or a 6?".

When we are writing the main program, we include the line that calls the function:

```
  firstday = aprilstartday();
```

At this stage we do not know the details of how the function will work. We simply know that it will provide us with the weekday code for 01 April in the given year. We can therefore continue designing and writing the main program without distractions. Using this function has simplified designing the

program, breaking the problem down into more manageable parts. When we have finished writing main, we can turn our full attention to what the function needs to do.

Functions can be used to isolate errors and thus assist with debugging

By using functions we can remove complicated sections of code (for example, how to read a test result and check that it is in the correct range) from main. In particular, functions allow us to focus on one small piece of the program and make sure that it is working correctly.

As an example, let us assume that we made a mistake in the `readmark` function. Let us say that we made a typo and entered the first line of the while loop as follows:

```
  while ((temp < 0) || (temp < maxmark)) {
```

Can you see what is wrong with this?

This causes a logic error. The program runs but it does not behave correctly. It will not accept correct test results and it will accept incorrect test results.

When we see this problem we realise that there is something wrong with the function because the task of the function is to read the test results. We can confirm that the problem is in the function by placing an extra `cout` in the function like this.

```
void readmark() {
  cout << "Enter the mark for the test ";
  cin >> temp;
  while ((temp < 0) || (temp < maxmark)) {
/**/ cout << "temp is " << temp << endl;     // an extra cout
    cout << "ERROR: mark must be from 0 to " << maxmark << endl;
    cout << "Enter the mark again ";
    cin >> temp;
  }
```

If we study the output from this extra `cout`, we will work out that the test for the loop is not correct. Then we can find the error and fix it.

All this debugging takes place inside the function. We leave main completely unchanged. In a big program, if we can isolate a logic error inside one function, it is much easier to find what is wrong than if we need to check the whole program.

Functions enable us to test the main program at an early stage

All programs need testing. However, testing becomes complicated if the entire program has to be completed before testing. At that late stage, it is difficult to determine which section of the program (or of a function) is causing the problem. Thus it is better to test the main program at an early stage and fix any errors that are occurring. Then the functions can be developed and tested separately later.

One approach is to write ***function stubs***. An example of a stub for the above program could be:

```
int aprilstartday() {
  return 1;   // Assume that April always starts on a Monday
}
```

A stub is a function that runs and returns a "reasonable" result but we don't have to spend a lot of time designing the function. Also the stub is very simple and so it is unlikely to have any errors in it. In this case the stub will work correctly for 2019 but not for most other years. That's ok, we will test the program only for 2019 (at this stage). The stub means that we can rapidly get to being able to run the

program and any errors that may occur are probably in main. We can then fix any errors that may be present in main before we get back to designing the full detail of the functions.

Functions make a program more readable (and thus easier to debug)

It is important that programs are easy to read. We write a program once but we read it many times (checking for errors) and other programmers may have to read it as well. If a program is easy to read, any errors in it will be quickly discovered. We have several ways to improve the readability of a program including:

- comments – good (but brief) comments explain the purpose of a function, statement etc
- indentation – this clearly shows which instructions are inside a loop, or a function, or… etc
- meaningful names for variables and functions
- the use of functions to break the problem into parts, reducing main to more of an overview of what the program is doing - something shorter and easier to follow

In the ANZAC Day program for example, if the two calls to `printday` were not there, we would have a whole sequence of instructions (twice) and main would be longer and harder to read.

`main` is a function

main is a function. It has a name, it has a return data type (`int`) and it has brackets (with no parameters). And main can have parameters as well but we won't get onto that in our course. Like all other functions, main can have local variables and can also access any global variables.